

...where:

- `{'student_list': student_list, 'flag': flag}` is the argument we have passed to the template file.
- The `flag` variable becomes true for URL `http://127.0.0.1:8000/helloweb/?flag` and false for `http://127.0.0.1:8000/helloweb/`.
- `{% if flag %}` is a conditional block that returns the code part by checking the Boolean value of the `flag` variable passed to the template.
- A `{% for name, rollno in student_list.items %}` statement is used to derive rows for the table from the data passed as the dictionary to the template file. This is the same as the Python statement: `for key, val in dictionary.items()`. Check out the following URLs, `http://127.0.0.1:8000/helloweb/?flag` and `http://127.0.0.1:8000/helloweb/` and see how they work.

Like the above template techniques, there are lots of options facilitated by the Django template system. The best place to look for more about Django templates is the code example from the Django documentation website at <http://docs.djangoproject.com/en/dev/ref/templates/builtins/>

Databases and models

Django is designed to work with different database management systems. It provides a nice interface to interact with the database- and storage-related tasks. Django does not use traditional SQL queries to interact with the database, like most of the other Web programming frameworks. It introduces a new concept called Models.

In Django, we do not need to work with any SQL queries; the data model concept introduced by Django makes it possible to handle data in terms of objects and groups. Therefore, we have the option of object-oriented database manipulation and management through Django. We manipulate every piece of data in terms of objects.

Let's go through a simple example on how to write a Web page that deals with the MySQL database. Create a database `helloweb_db`, using MySQL or phpmyadmin, with the user name `helloweb` and password `hellowebpass`.

To run applications involving MySQL access, make sure that the `mysql-server` and `python-mysqldb` packages are already installed. Modify `settings.py` with the MySQL database details as explained in the earlier part of this article.

Django can only handle one project at a time. But it can execute many applications. Applications are sub-modules of the Django project. While deploying the Web server, we set the Web root directory to a Django project. We access all other Django applications related to that project by postfixing a path to the Web root URL.

Now we will create a Django application to learn how to code database interactive pages.

```
slynn@hacking:~/LPY/helloweb$ ./manage.py startapp student
slynn@hacking:~/LPY/helloweb$ cd student
```

Now the skeleton files for the application book appear in

the `book/` directory. There will be a `models.py` file. Write the following code in it:

```
from django.db import models

class StudentRegister(models.Model):
    name = models.CharField(max_length=30)
    guardian_name = models.CharField(max_length=30)
    rollno = models.IntegerField()
    admission_date = models.DateTimeField()
```

We now need to add the reference to the application we created to the `INSTALLED_APPS` tuple of the `settings.py` file—i.e., add the entry `helloweb.student` to the tuple.

To create the corresponding tables automatically in the database, issue the following command:

```
slynn@hacking:~/LPY/helloweb$ ./manage.py syncdb
Creating table auth.permission
Creating table auth_group
Creating table auth_user
Creating table auth_message
Creating table django_content_type
Creating table django_session
Creating table django_site
Creating table student_studentregister
```

You just installed Django's `auth` system, which means you don't have any superusers defined.

```
Would you like to create one now? (yes/no): yes
Username (leave blank to use 'slynn'):
```

```
Username (leave blank to use 'slynn'):
```

```
E-mail address: slynn@slynn.com
```

```
Password:
```

```
Password (again):
```

```
Superuser created successfully.
```

```
Installing index for auth.Permission model
```

```
Installing index for auth.Message model
```

You can see that besides the `student_studentregister` tables, so many other tables are also created. These are actually for the user administration option provided by Django. For every standard Web application, there will be an administration page. If you have used any content management system like Drupal, you would have seen some kind of administration page that can handle many users, the database contents, etc. Django has a nice feature — by default, it comes with basic data model administration. We can customise this to handle many users and add additional features. You can learn more about the administration interface at <http://docs.djangoproject.com/en/dev/intro/tutorial02/#activate-the-admin-site>.

You can view the SQL code used by Django internally to create tables by issuing the following command:

```
slynn@hacking:~/LPY/helloweb$ ./manage.py sql student
BEGIN;
CREATE TABLE 'student_studentregister' (
```